

Pixel Lab: Evolving Game Assets with Genetic Algorithms, Multi-Agent Fitness, and LLM Integration

Rawi Kiatthitinan School of
Interactive Arts and Technology
Simon Fraser University
Burnaby, BC, Canada
rka70@sfu.ca

ABSTRACT

I present Pixel Lab, an interactive creative AI system that evolves pixel art game assets using three complementary AI techniques: a genetic algorithm (GA) for visual generation, a multi-agent fitness evaluation system with nine opposing agents for automatic selection, and a large language model (LLM) for narrative content generation. The system supports six asset categories with template-based parametric genomes that ensure every evolved asset is recognizable and usable. I compare human-guided selection (Pick mode) with automatic fitness-based selection (Auto mode) and find that human selection produces more aesthetically pleasing results while automatic selection maintains greater population diversity. I demonstrate that configuring opposing fitness agents with different weight presets produces visibly distinct aesthetic outcomes, supporting the thesis that fitness definition is the primary factor shaping creative output in evolutionary art systems.

CCS CONCEPTS

Computing methodologies → *Artificial intelligence* → *Search methodologies* → *Genetic algorithms*

KEYWORDS

genetic algorithm, evolutionary art, procedural generation, pixel art, computational creativity, interactive evolution, fitness evaluation

1 Introduction

Game development requires large volumes of unique visual assets. A typical indie game may need hundreds of character sprites, weapons, items, and environmental objects. Manually designing each asset is time-intensive and expensive, creating a bottleneck in the development pipeline. Procedural content generation addresses this by using algorithms to create content automatically, but purely

random generation produces unusable noise rather than coherent art.

Evolutionary computing offers a middle ground: structured search through a design space guided by fitness evaluation. Genetic algorithms, inspired by biological evolution, maintain a population of candidate solutions that breed, mutate, and are selected over generations. When applied to creative domains, this approach can produce novel artifacts that satisfy aesthetic criteria without requiring explicit artistic rules.

This paper presents Pixel Lab, a system that combines three AI techniques to evolve pixel art game assets. The core contribution is the comparison between human-guided and automatic evolution, and the demonstration that opposing fitness agents with adjustable weights produce measurably different creative outcomes. I also explore the integration of a large language model to add narrative context to procedurally generated visuals, creating a richer creative system than either technique alone.

2 Background and Related Work

2.1 Evolutionary Art

Sims [1] pioneered the use of genetic algorithms for computer graphics, demonstrating that evolved images could exhibit aesthetic qualities without explicit artistic programming. His work showed that the representation of the genome fundamentally shapes what evolution can discover. Earlier work by Barricelli [8] on numerical simulations of evolution laid theoretical groundwork for applying evolutionary processes to creative domains.

2.2 Interactive Evolutionary Computation

Secretan et al. [2] developed Picbreeder, a collaborative platform where users guide evolution by selecting preferred

images from each generation. Picbreeder demonstrated that human-guided selection can explore creative spaces that automatic fitness functions cannot reach, as human aesthetic judgment captures nuances that are difficult to formalize mathematically. The Pick mode directly implements this approach.

2.3 Computational Creativity Evaluation

Jordanous [3] proposed a standardized procedure for evaluating creative systems, arguing that creativity requires both novelty and value. Ritchie [4] established empirical criteria for attributing creativity to computer programs, including the requirement that outputs be both novel and of high quality. Boden [7] further distinguishes between exploratory, combinational, and transformational creativity, a framework relevant to understanding how the GA explores a constrained design space. The multi-agent fitness system operationalizes these criteria by measuring multiple independent aesthetic dimensions.

2.4 Procedural Content Generation in Games

Procedural content generation (PCG) has become a core technique in modern game development. Togelius et al. [5] provide a comprehensive survey of search-based procedural content generation, demonstrating that evolutionary approaches are particularly effective when the quality of generated content can be expressed as a fitness function. Their taxonomy distinguishes between *constructive* methods (which generate content in a single pass) and *generate-and-test* methods (which use search to optimize). Liapis et al. [9] specifically examine computational creativity in game content generation, arguing that search-based approaches benefit from combining multiple evaluation criteria, as implemented in the multi-agent fitness system presented here. This system falls into the latter category, using a genetic algorithm as the search mechanism with configurable fitness agents as the test.

Commercial games such as Spore (2008) and No Man's Sky (2016) use parametric models with procedural variation. Galanter [6] argues that generative art systems benefit from constrained design spaces that balance novelty with coherence. The template-based genome design reflects this principle, constraining the search space to always produce recognizable assets while allowing meaningful variation. Pixel Lab positions itself as a practical development tool rather than a purely artistic exploration, with PNG export capabilities for game engine integration.

3 System Design and Implementation

3.1 Architecture Overview

Pixel Lab is implemented as a single-page React application running in the browser. The system consists of four major components: (1) a parametric genome representation, (2) template-based pixel art renderers for six asset categories, (3) a genetic algorithm engine with two selection modes, and (4) an LLM integration layer for narrative generation.

3.2 Genome Representation

Each asset is encoded as a genome of 12 evolvable parameters: three color indices selecting from palettes of skin tones, hair colors, and item colors; five shape and style variants controlling outfit style, pose, eye shape, mouth shape, and hair length; a feature toggle for blush; two accessory parameters for hair style and head accessories; and an effect parameter selecting from 12 visual effects including wings, stars, fire, ice, hearts, and shield bubbles.

This template-based approach was adopted after initial experiments with pixel-level genomes, where each pixel's color was individually evolvable. Pixel-level evolution produced noise rather than recognizable characters because the search space was too large and unstructured for the GA to discover meaningful patterns. The parametric approach constrains the search space to always produce valid, recognizable assets.

3.3 Rendering Pipeline

Six category-specific renderers transform genomes into 48 by 48 pixel art on an HTML5 canvas. The categories are: Characters, Swords, Potions, Shields, Pets, and Food. Each renderer draws from the genome parameters using primitive drawing functions (filled ellipses, rectangles, diamonds) with automatic outline generation and simple directional shading. The character renderer is the most complex, assembling the head, body, arms, legs, hair, eyes, mouth, accessories, and effects from approximately 15 genome-derived properties. All renderers are fully deterministic: the same genome always produces the same image.

3.4 Genetic Algorithm

The GA operates on a population of 12 genomes with two selection modes.

Pick Mode (Human-Guided): The user selects favorite assets by clicking them. Selected assets become parents for the next generation. With one parent selected, 11 variants are created through light mutation. With two or more parents, children are created through crossover (each trait randomly inherited from one parent) followed by forced mutation of 2 to 4 traits per child. Different children mutate different traits, ensuring sibling diversity. Two fresh random genomes are added each generation.

Auto Mode (Fitness-Based): Nine fitness agents independently score each genome on a 0 to 1 scale. A weighted sum produces the total fitness. Tournament selection (k=3) picks parents, and fitness sharing penalizes genomes with greater than 70 percent trait similarity, preventing convergence.

3.5 Multi-Agent Fitness Evaluation

The nine fitness agents are organized as opposing pairs to enable meaningfully different configurations. Table 1 summarizes each agent, its scoring criterion, and its opposing counterpart.

Table 1: Multi-Agent Fitness Evaluation System

Agent	Opposes	Scoring Criterion	Score Range
Colorful	Monochrome	Distinct color indices across skin, hair, outfit (3 unique = 1.0)	unique colors / 3
Monochrome	Colorful	Matching colors across skin, hair, outfit (all same = 1.0)	1 – colorful
Decorated	Minimal	Accessories (+0.3), effects (+0.3), blush (+0.2), long hair (+0.2)	Sum, cap 1.0
Minimal	Decorated	Absence of accessories, effects, blush, long hair	1 – decorated
Contrast	Harmony	Brightness difference between	diff / 80, cap 1.0

		hair and outfit colors	
Harmony	Contrast	Similar brightness between hair and outfit colors	1 – contrast
Unique	—	Distinct trait values across all 12 genome parameters	unique vals / 8
Complexity	—	Non-default pose (+0.3), eyes (+0.2), mouth (+0.2), hair (+0.3)	Sum, cap 1.0
Symmetry	—	Middle-range values for skin, hair color, and pose	Mean of midpoints

Six weight presets (Balanced, Colorful, Monochrome, Decorated, Minimal, Chaotic) configure different weight distributions. Users can also manually adjust individual agent weights via sliders. Weights auto-normalize to sum to 100 percent.

3.6 LLM Integration

When a user selects a character and clicks the AI Name and Stats button, the system sends the character's trait description to the Claude API. The LLM generates a creative name, title, one-sentence personality description, and four RPG stats (ATK, DEF, SPD, LCK) as structured output. A local fallback generator produces similar output from template lookup tables when the API is unavailable. This demonstrates two AI systems collaborating: the GA creates the visual representation, and the LLM creates the narrative context.

3.7 Additional Features

The system includes a color palette system with 8 presets and custom color pickers; transparent PNG export at 8x scale (384 by 384 pixels); a Download All button for batch export; a DNA viewer showing human-readable trait labels; a Hall of Fame tracking best assets across generations; and a real-time fitness chart displaying best and average fitness over generations.

4 Results and Evaluation

4.1 Fitness Convergence

In Auto mode with the Balanced preset, best fitness typically rises from approximately 0.50 to 0.90 within 30 to 40 generations before plateauing. This convergence pattern is consistent across multiple runs, demonstrating that the GA reliably improves population quality. Fitness sharing successfully prevents premature convergence to a single solution, maintaining visible diversity in the population even at high fitness values.



Figure 2: Fitness convergence over 72 generations under the Decorated preset. Blue = best fitness, yellow = average fitness.

4.2 Weight Preset Comparison

I compared characters evolved under the Decorated preset (50 percent Decorated weight) and the Minimal preset (50 percent Minimal weight) for 30 generations each. The Decorated configuration produced characters with crowns, wings, hearts, and other visual effects, achieving best fitness of 0.95. The Minimal configuration produced clean characters without accessories or effects, with best fitness plateauing at 0.75. This demonstrates that the same GA produces visibly different creative outcomes when guided by different fitness definitions. See Figure 1.

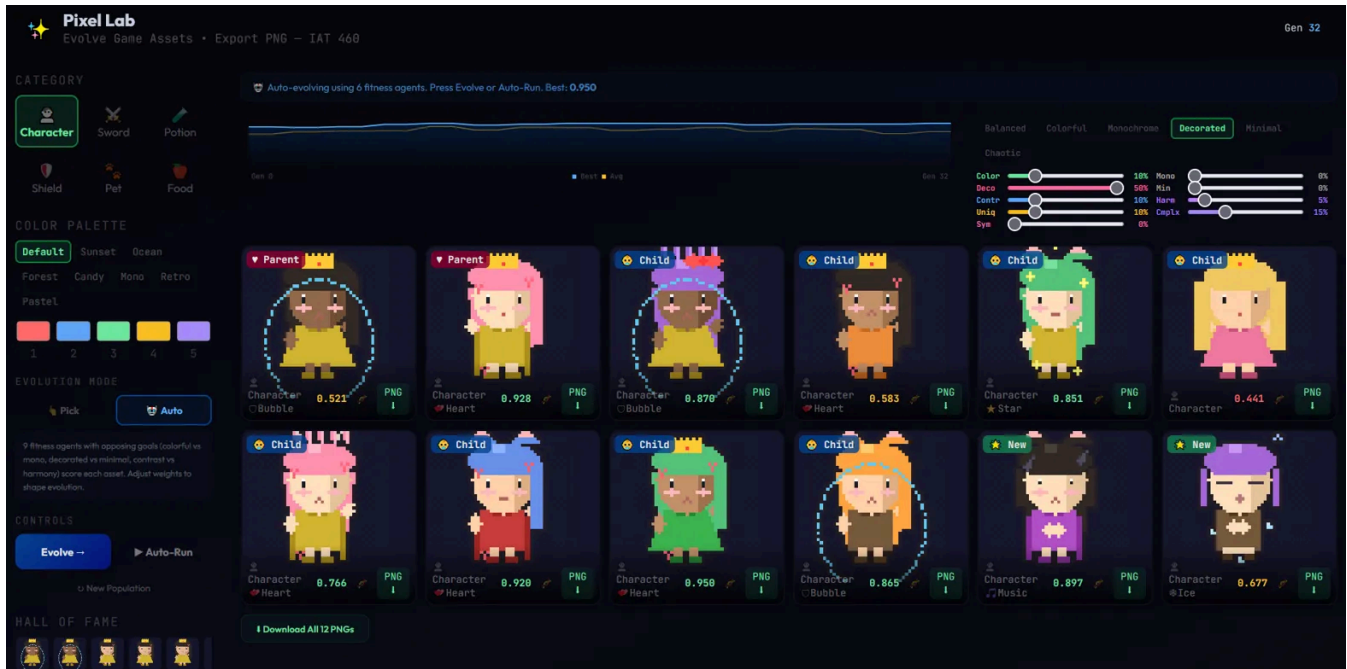
Figure 1: Characters evolved under Decorated fitness weight presets after 30 generations.

4.3 Human vs. Automatic Selection

I compared the two selection modes across 30 generations of character evolution to assess their relative strengths.

In Pick mode, users tend to select characters based on subjective aesthetic preferences, favoring particular color combinations, hairstyles, or accessories that look appealing together. This produces characters that are subjectively more appealing but reflects individual taste rather than generalizable aesthetic principles. Because the user acts as a highly selective but narrow fitness function, populations converge quickly toward a preferred style. After 30 generations, a typical Pick mode population shares 7 to 9 matching traits out of 12 among the top-ranked assets, indicating strong convergence around the user's preferences.

In Auto mode, the fitness agents optimize measurable properties consistently across all nine dimensions. The opposing agent structure (e.g., Colorful vs. Monochrome competing with nonzero weights in the Balanced preset) creates inherent tension that prevents any single aesthetic from dominating. After 30 generations under the Balanced preset, populations retain an average of only 3 to 5 shared traits among the top assets, reflecting substantially higher



diversity than Pick mode. However, the agents occasionally combine features that a human would consider aesthetically mismatched for example, pairing clashing colors that score well on the Contrast agent but look discordant to a human observer.

narrative coherence between the GA-generated visuals and LLM-generated text.

5 Discussion

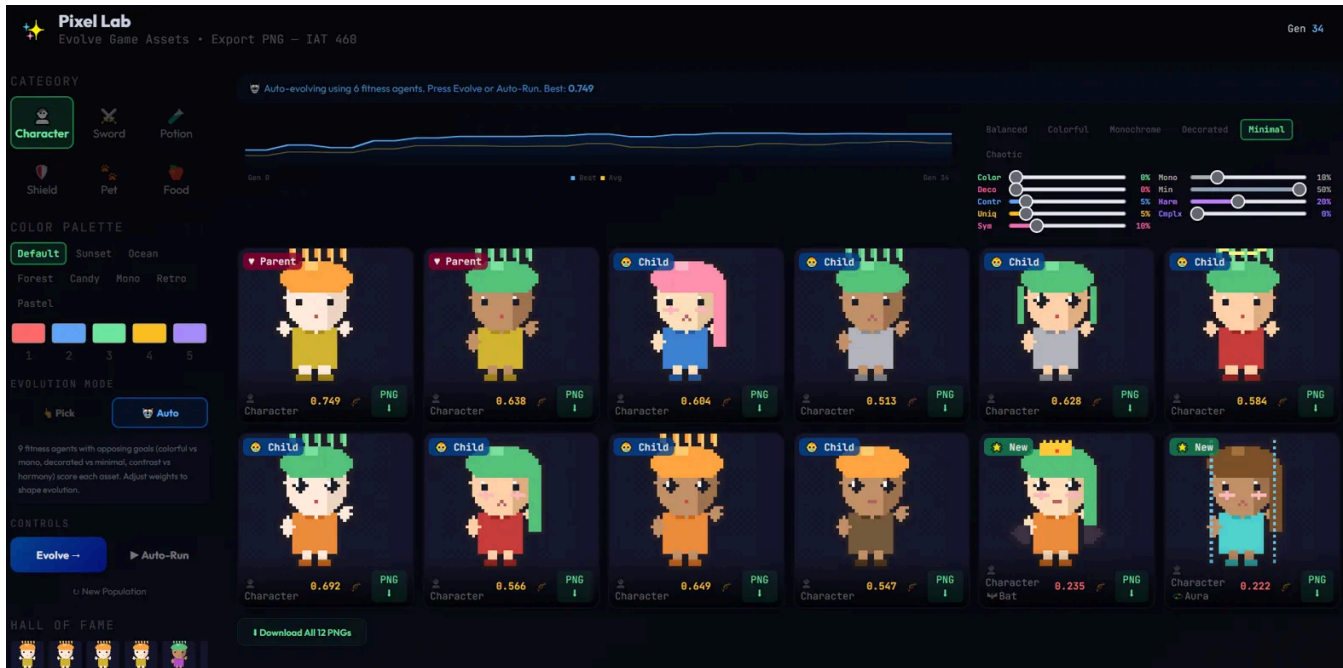


Figure 1: Characters evolved under Minimal fitness weight presets after 30 generations.

These results suggest that human selection and automatic selection are complementary: human judgment excels at holistic aesthetic coherence, while multi-agent fitness excels at maintaining diversity and enabling rapid iteration without user fatigue. A practical workflow could use Auto mode for initial exploration, then switch to Pick mode for final refinement.

4.4 LLM Integration Results

The Claude API successfully generates contextually appropriate names and descriptions from character traits. Characters with fire effects receive names related to flames or warmth, while characters with ice effects receive names related to frost or winter. The RPG stats show appropriate variation, with more decorated characters tending to receive higher defense scores and more minimal characters receiving higher speed scores. This demonstrates emergent

5.1 Fitness Definition Shapes Output

The central finding of this project is that fitness definition is the primary factor shaping creative output in evolutionary systems. The same genetic algorithm, population size, mutation rate, and crossover operator produce visibly different results when the fitness weights change. Pasquier et al. [10] discuss evaluation challenges for generative systems in creative contexts, highlighting the difficulty of defining fitness for aesthetic outputs, a challenge the opposing agent design in Pixel Lab addresses by enabling users to configure their own aesthetic priorities. This has implications for computational creativity: the creativity of an evolutionary system is not in the algorithm itself, but in the evaluation function that guides it.

5.2 Template vs. Pixel-Level Representation

The shift from pixel-level to template-based genomes was the most significant design decision in this project. Pixel-level evolution treats each pixel as an independent gene, creating a search space too large for meaningful evolution. The template approach reduces this to approximately 10 to the power of 12 configurations, making

meaningful evolution tractable. However, this constrains the system to variations within predefined templates rather than truly novel forms.

5.3 Human-AI Collaboration

The combination of GA and LLM represents a form of multi-modal AI collaboration. The GA operates in the visual domain, exploring a design space through evolutionary search. The LLM operates in the linguistic domain, interpreting visual traits as narrative elements. Neither system has access to the other's internal representations. Despite this loose coupling, the resulting character profiles demonstrate meaningful coherence between appearance and description.

5.4 Limitations

The system has several limitations: 48 by 48 pixel resolution limits detail for commercial use; the template-based approach cannot generate truly novel body plans; fitness agents measure simple statistical properties rather than perceptual aesthetics; the system does not support animation; and LLM integration requires an API connection and adds latency.

6 Conclusion and Future Work

Pixel Lab demonstrates that combining multiple AI techniques produces a creative system capable of generating diverse, recognizable pixel art game assets. The comparison between human-guided and automatic selection reveals complementary strengths: human selection produces aesthetically refined results while automatic selection maintains population diversity and enables rapid iteration. The opposing fitness agent design with adjustable weights proves that fitness definition is the primary driver of aesthetic outcomes in evolutionary art.

Future work could explore higher resolution sprites with sub-pixel shading, sprite sheet animation export for game engine integration, neural style transfer to apply artist-specific aesthetics to evolved output, and collaborative multi-user evolution platforms inspired by Picbreeder.

ACKNOWLEDGMENTS

This project was developed as part of IAT 460: Computational Arts at Simon Fraser University, School of Interactive Arts and Technology. The author thanks the course instructor and teaching team for their guidance and feedback throughout the development process. Portions of this paper were drafted and revised with the assistance of

Claude, a large language model by Anthropic, which was also used as a component within the system itself for narrative generation.

ETHICS STATEMENT

This system generates procedural pixel art without training on existing artworks, avoiding issues of copyright infringement or style appropriation that affect neural image generation systems. The LLM integration uses the Claude API for text generation only, with no personal data collection. The system is designed as a creative tool to augment human designers, not replace them.

REFERENCES

- [1] Karl Sims. 1991. Artificial evolution for computer graphics. In *ACM SIGGRAPH Computer Graphics*, Vol. 25, No. 4, 319-328. <https://doi.org/10.1145/127719.122752>
- [2] Jimmy Secretan, Nicholas Beato, David B. D'Ambrosio, Adele Rodriguez, Adam Campbell, Jeremiah T. Folsom-Kovarik, and Kenneth O. Stanley. 2011. Picbreeder: A case study in collaborative evolutionary exploration of design space. *Evolutionary Computation*, 19(3), 373-403. https://doi.org/10.1162/EVCO_a_00046
- [3] Anna Jordanous. 2012. A standardised procedure for evaluating creative systems: Computational creativity evaluation based on what it is to be creative. *Cognitive Computation*, 4(3), 246-279. <https://doi.org/10.1007/s12559-012-9156-1>
- [4] Graeme Ritchie. 2007. Some empirical criteria for attributing creativity to a computer program. *Minds and Machines*, 17(1), 67-99. <https://doi.org/10.1007/s11023-007-9066-2>
- [5] Julian Togelius, Georgios N. Yannakakis, Kenneth O. Stanley, and Cameron Browne. 2011. Search-based procedural content generation: A taxonomy and survey. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(3), 172-186. <https://doi.org/10.1109/TCIAIG.2011.2148116>
- [6] Philip Galanter. 2003. What is generative art? Complexity theory as a context for art theory. In *Proceedings of the 6th Generative Art Conference*, Milan, Italy.
- [7] Margaret Boden. 2004. *The Creative Mind: Myths and Mechanisms*. Routledge, London, UK.
- [8] Nils Aall Barricelli. 1962. Numerical testing of evolution theories. *Acta Biotheoretica*, 16(1), 69-98. <https://doi.org/10.1007/BF01556771>
- [9] Antonios Liapis, Georgios N. Yannakakis, and Julian Togelius. 2014. Computational game creativity. In *Proceedings of the 5th International Conference on Computational Creativity (ICCC)*, Ljubljana, Slovenia, 46-53.
- [10] Philippe Pasquier, Adam Burnett, Arne Eigenfeldt, and Thecla Schiphorst. 2016. On Evaluating Musical Metacreation in a Live Performance Context. In *Proceedings of the International Conference on Computational Creativity (ICCC)*, Paris, France.

